

CS 320 Project Two: Summary and Reflections Report

Southern New Hampshire University

CS 320: Software Test, Automation, and Quality Assurance

Hetal Ram

Professor Jonathan Norman

August 10, 2026

Summary

Unit Testing Approach and Alignment to Requirements

For Project One, I developed my unit testing approach directly from the software requirements for the Contact, Task, and Appointment features. I used JUnit 5 to test both valid and invalid inputs. My goal was not only to make sure that valid information was accepted, but also to verify that invalid information was rejected correctly. I also tested the service classes to make sure the required add, delete, and update operations worked as expected.

For the Contact feature, I tested the requirements for the contact ID, first name, last name, phone number, and address. ContactTest checks invalid conditions such as a null contact ID and an ID longer than 10 characters. I also tested null and overlength first and last names, an invalid phone number of length, and null or overlength addresses. In ContactServiceTest, I tested adding and deleting contacts, updating the allowed fields, and preventing duplicate contact IDs. For example, lines 52–64 update the first name, last name, phone number, and address and then use assertEquals to confirm that each change was successful.

For the Task feature, I followed the same requirement-based approach. The task ID cannot be null or longer than 10 characters, the name cannot be null or longer than 20 characters, and the description cannot be null or longer than 50 characters. After receiving feedback on my Module Four milestone, I realized that my original TaskTest did not test every validation requirement. I improved the test class by adding the missing tests for null values and maximum field lengths. For example, lines 21–23 test an 11-character task ID and expect an IllegalArgumentException. TaskServiceTest also verifies adding, deleting, and updating tasks and rejecting duplicate task IDs.

For the Appointment feature, I tested the appointment ID, appointment date, and description. An important requirement was that the appointment date could not be in the past. In

AppointmentTest, lines 45–50 create a past date and use assertThrows to verify that the date is rejected. I also tested a null date, null ID, an ID longer than 10 characters, a null description, and a description longer than 50 characters. AppointmentServiceTest verifies adding and deleting appointments and also tests duplicate IDs, null appointments, missing appointment IDs, and null IDs.

Overall, my testing approach was strongly aligned with the software requirements because I converted the individual requirements into specific JUnit test cases. Instead of testing only successful situations, I included invalid and boundary conditions to make sure the classes and services behaved correctly when incorrect data was provided.

Effectiveness of the JUnit Tests

I used Eclipse code coverage to evaluate the effectiveness of my JUnit tests. My results showed that all six implementation files met the Project One requirement of at least 80% coverage.

Contact.java reached 86.5% coverage and ContactService.java reached 81.7%. Task.java reached 88.1%, while TaskService.java reached 80.3%. Appointment.java and AppointmentService.java both reached 100% coverage.

These results gave me confidence that my tests executed most of the important application logic.

At the same time, I learned that a high coverage percentage does not automatically mean that every possible defect has been found. The quality of the assertions and the test cases is also important. Louridas (2005) discusses the value of unit testing for finding and correcting defects close to the phase in which they are introduced. In my project, I combined coverage with positive

tests, negative tests, boundary conditions, and exception testing to provide stronger evidence that the software requirements were being verified.

Technically Sound Code

While writing my JUnit tests, I tried to keep each test focused on a particular requirement and use an assertion that matched the expected result. This helped me determine exactly what behavior was being tested and made failures easier to understand.

For example, lines 32–38 of `TaskServiceTest` add a task, update its fields, retrieve the task, and use `assertEquals` to verify that the name changed to "Project" and the description changed to "New Description". In `AppointmentServiceTest`, lines 39–41 use `assertThrows(IllegalArgumentException.class, ...)` to verify that an appointment with a duplicate ID cannot be added. I also used `assertNull` after deletion operations to verify that deleted objects were no longer available.

These tests helped make the code technically sound because they checked actual outcomes rather than simply executing methods. Valid inputs were checked for correct results, while invalid inputs were checked for the expected exceptions.

Efficient Code

I also tried to keep my tests efficient and easy to understand. Each test creates only the objects needed for that particular situation. For example, lines 17–24 of `TaskServiceTest` create a service and a task, add the task, delete it, and use `assertNull` to verify that the task has been removed.

There are no unnecessary loops, database connections, or user interface operations in these tests. The Contact, Task, and Appointment services use in-memory data structures, which allows the tests to run quickly. Keeping each test focused on one behavior also makes maintenance easier because if a test fails, I can identify the related requirement more quickly.

Reflection

Testing Techniques Employed

The main software testing technique I used in Project One was unit testing with JUnit 5. I also used positive testing, negative testing, boundary testing, and exception testing.

Positive testing was used to verify that valid data produced the expected result. For example, `testValidTask()` creates a task containing a valid ID, name, and description and verifies those values using `assertEquals`.

Negative testing was used to verify that invalid values were rejected. Examples included null values, overlength fields, duplicate IDs, and past appointment dates. Boundary testing was especially useful for requirements involving maximum field lengths. For example, because a task ID cannot contain more than 10 characters, I tested an 11-character ID to verify that it was rejected.

Exception testing was also important because many invalid conditions were expected to result in an `IllegalArgumentException`. Using `assertThrows` allowed me to verify that the application rejected invalid data in the expected way. Hambling et al. (2019) describe systematic test techniques as an important part of designing tests and evaluating software behavior.

Other Testing Techniques and Their Practical Uses

There are several testing techniques that I did not use in Project One, including integration testing, system testing, performance testing, usability testing, and security testing.

Integration testing would test how different components work together. For example, if the Contact, Task, or Appointment services were connected to a database or external API, integration testing could verify that information moves correctly between those components. System testing evaluates the complete application as a whole rather than individual classes.

Performance testing examines how an application behaves under heavier workloads and would be important for an application serving many users at the same time. Usability testing evaluates how easily users can understand and interact with an application. Because Project One did not include a user interface, usability testing was not necessary. Security testing identifies vulnerabilities and would become especially important if the application stored personal or sensitive customer information.

The practical use of each technique depends on the type, complexity, and risks of the software being developed. Unit, boundary, negative, and exception testing were appropriate for Project One because the project focused on individual Java classes and in-memory services. Integration testing would become more important if these services were connected to other systems, while system, performance, usability, and security testing would become more important as the application developed into a complete production system. This helped me understand that testing strategies should be selected according to the requirements and risks of a project rather than using the same testing technique in every situation (Hambling et al., 2019).

Caution

I learned that caution is an important part of the software tester mindset. Code may work correctly with normal input while still failing at its boundaries or when unexpected data is provided. For example, a task ID containing 11 characters is only one character beyond the allowed limit, but accepting it would still violate the customer's requirement. Similarly, allowing an appointment date in the past would violate an important Appointment requirement.

I also had to consider the relationships between the object classes and their service classes. The Contact, Task, and Appointment objects validate their information, while the corresponding service classes manage those objects through operations such as adding, deleting, and updating. A problem in either area could affect the overall behavior of the application. Testing both the object and service classes helped me evaluate these different responsibilities carefully.

Limiting Bias

Bias can be a concern when developers test their own code because they already know what they expect the program to do. This can make it easy to concentrate on valid inputs and overlook situations that might cause the software to fail.

I tried to reduce this bias by using the written requirements as the basis for my tests rather than relying only on my assumptions about the code. My Module Four milestone feedback was a good example. My original TaskTest did not verify every validation requirement. After reviewing the instructor's feedback, I added tests for null ID, null name, null description, overlength ID, overlength name, and overlength description.

This experience showed me why independent review and requirement-based testing are valuable. Even when code appears to work correctly, a developer can overlook an important test condition. Using requirements, instructor feedback, and coverage results helped me look at the code more objectively.

Discipline and Commitment to Quality

Being disciplined about software quality is important because cutting corners during development and testing can create technical debt. A small validation issue that is ignored today can become a larger problem later when additional features depend on the same code. Fixing defects after software has become more complex can also require more time and effort.

As a software engineering professional, I plan to reduce technical debt by keeping tests connected to requirements, running tests whenever code changes, maintaining readable test code, reviewing coverage results, and adding regression tests when defects are discovered. On larger projects, I would also use code reviews and automated continuous integration testing so that changes are tested regularly.

Project One taught me that software quality is not simply about getting a program to compile or run. Quality requires discipline throughout development and testing. A developer must verify valid behavior, consider invalid and boundary conditions, review results carefully, and continue testing when the software changes. Following these practices can help produce software that is more reliable, maintainable, and consistent with customer requirements.

References

Hambling, B., Morgan, P., Samaroo, A., Thompson, G., & Williams, P. (2019). *Software testing: An ISTQB-BCS certified tester foundation guide* (4th ed.). BCS Learning & Development Limited. <https://ebookcentral.proquest.com/lib/snhu-ebooks/detail.action?docID=5837074>

Louridas, P. (2005). JUnit: Unit testing and coding in tandem. *IEEE Software*, 22(4), 12–15.

<https://research.ebsco.com/c/ix3dnl/search/details/6mbo4plxqr?q=junit>